CSA02 C Programming – CO1 Assignment Answers

Assignment 1: Rural Electricity Co-operative Billing

Note: These are model answers based on the uploaded assignment sheets. Replace student-specific details and add your own compiler screenshots, timing measurements and GitHub links before submission.

Task 1 – Problem analysis

Inputs: household ID/category, previous and current 16-bit meter readings. Derived units = (current - previous) modulo 65536. Outputs: units, slab energy charge, fixed charge and total. Constants: meter modulus 65536, slab boundaries 100 and 200, rates 0/225/450 paise and fixed charge 5000 paise. Rules: units and bill cannot be negative; total = fixed + energy; all money is exact to paise. Assumptions: at most one rollover between readings; a reading is valid in 0..65535; rates are applied to whole units.

Text readings require parsing and validation; conversion and comparison take O(d) time for d digits and store d characters, while an integer reading uses fixed O(1) space and arithmetic.

Task 2 – Data types

Use unsigned short for each 16-bit meter register (verify USHRT_MAX == 65535); unsigned int or uint32_t for consumption; unsigned long long for aggregate units; long long for money in paise. Use char arrays for IDs/categories. Avoid float for money because binary floating point cannot represent many decimal fractions exactly. Store Rs 2.25 as 225 paise.

Example: adding 0.1 as float repeatedly may produce a value slightly different from the mathematical total; integer paise additions remain exact provided the chosen integer type does not overflow. For 4,000 bills, use a 64-bit accumulator.

Task 3 – Constants

#define is suitable for compile-time tariff macros; const long values are typed and visible to a debugger; enum is suitable for integral symbolic categories. Keep all rates in one header. A tariff change then requires one edit. Prefer const variables for typed configuration and #define for include guards and conditional compilation.

Task 4 – Safe macros

Correct forms: #define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL), #define MAX(a,b) (((a) > (b)) ? (a) : (b)), #define SQUARE(x) ((x) * (x)). Parentheses protect precedence, but MAX(i++,j) can evaluate i++ twice; use a static inline function when arguments may have side effects. gcc -E file.c shows macro expansion.

Task 5 – Header files and DEBUG

billing.h should contain include guards, constants, typedefs and prototypes. Use #include for system headers and #include "billing.h" for project headers. Example: #ifdef DEBUG / fprintf(stderr,"units=%u\n",units); / #endif. With -DDEBUG the diagnostic code is compiled in; without it the code is omitted. Runtime debug flags retain code and perform a condition check.

Task 6 – Input/output

Use matching formats: %hu for unsigned short, %u for unsigned int, %lld for long long, %ld for long, %s for bounded strings and %c for a character. For scanf, pass addresses (e.g. scanf("%hu", &reading;)); check the return count. Add field widths such as %19s to avoid buffer overflow. Print money as paise/100 and paise%100 using integer formats.

Task 7 – Type conversion and precision

Promote meter operands to unsigned arithmetic of sufficient width before subtraction. Avoid mixing signed and unsigned values without explicit conversion. Compute charges in long long paise: each multiplication is promoted before it occurs. Explicitly cast before division when fractional output is required; integer division truncates. Use integer arithmetic throughout billing.

Task 8 – Compilation

gcc -E billing.c -o billing.i performs preprocessing; gcc -S billing.c -o billing.s emits assembly; gcc -c billing.c -o billing.o assembles to an object file; gcc billing.o -o billing links the executable. Include -Wall -Wextra during compilation. The linker resolves external symbols and library calls.

Task 9 – Implementation and tests

```
#include <stdio.h>\n#include <limits.h>\ntypedef unsigned short meter_t;\n#define S1 100\n#define S2 200\n#define R1 0LL\n#defir
```

Expected slab values: 0 and 100 units = Rs 50.00; 150 = Rs 162.50; 200 = Rs 275.00; 350 = Rs 950.00. Rollover example 65400 to 120 gives 256 units. Test zero, both boundaries, just above boundaries, rollover and invalid input.

Task 10 – Expectations vs execution

Compare expected and measured sizes with sizeof and limits with . Compare float totals against paise totals for 4,000, 60,000 and 800,000 bills. Record compiler/platform and observed differences; do not claim measured results without running the program. The integer-paise version should match exact arithmetic.

Task 11 – Final recommendation

Use unsigned short for hardware readings, unsigned int/uint32_t for per-period consumption, uint64_t or unsigned long long for large aggregates, and signed 64-bit paise for money. Centralize tariff definitions, validate input, use bounded strings and compile with warnings. This provides exact currency and headroom for district scale.

Task 12 – Smart-meter redesign

At 15-minute intervals, a household can produce 35,040 readings/year. Store watt-hours or smaller integer energy units in uint64_t, with timestamps and tariff-period identifiers. A fixed #define tariff is no longer suitable if the board changes tariffs without recompilation: load validated tariff configuration from a file/database. Apply a 20% peak premium from 18:00–22:00 using integer/rational arithmetic (e.g. base_paise * 120 / 100 with a defined rounding rule). Use 64-bit totals and test maximum yearly consumption, daylight/time boundaries, missing intervals and tariff revisions. Modify rather than retain the original simple monthly design.